Running the Lava Local Stack

Field	Value
Document	`docs/guides/local-stack-setup.md`
Revision	1
Last updated	2026-06-08
Status	Current
Scope	How to run `lava-api-go` locally via the Containers submodule and have the Android app auto-discover it over mDNS

> Grounded in `start.sh`, `stop.sh`, `docker-compose.yml`, `lava-api-go/internal/config/config.go`, and `docs/LOCAL_NETWORK_DISCOVERY.md`. Items that are not yet in the tree are marked PENDING (§11.4.6 — no guessing).

What the local stack is

The Lava backend is **`lava-api-go`** — a Go service that scrapes the upstream trackers and exposes a JSON API. The Android app can talk to a `lava-api-go` instance running on your own LAN, and it can **find that instance automatically** over mDNS. There is no cloud dependency for this path.

> **Note:** the legacy Ktor `:proxy` server was removed on 2026-05-06 (see the header comment in `start.sh` and `docker-compose.yml`). Some older text in `docs/LOCAL_NETWORK_DISCOVERY.md` still mentions the proxy fat JAR, port 8080, and the `_lava._tcp` service type — those refer to the retired proxy. The current backend is `lava-api-go`, listening on `:8443` and advertising `_lava-api._tcp`.

Starting the stack

The single entry point is `./start.sh` at the repo root. It delegates to the Lava-domain CLI at `tools/lava-containers/`, which auto-detects Podman or Docker and brings the service up via `docker-compose.yml`.

```
./start.sh                              # api-go profile only
        (default)
./start.sh --with-observability         # + Prometheus / Loki /
        Promtail / Tempo / Grafana
./start.sh --dev-docs                   # + Swagger UI
```

```
./start.sh --with-observability --dev-docs   # combine
./start.sh --help                            # usage
```

What `start.sh` does, in order (read from the script):

1. **Builds the `lava-containers` CLI** if its binary is missing (`tools/lava-containers/bin/lava-containers`, built with `go build`).
2. **Provisions TLS material** for the HTTPS / HTTP-3 listener if absent, by running `lava-api-go/scripts/gen-cert.sh` (creates `lava-api-go/docker/tls/server.crt` + `server.key`).
3. **Ensures a Postgres password** — appends a deterministic LAN-only default `LAVA_PG_PASSWORD` to `.env` if you have not set one (the `api-go` profile's compose stanza requires it).
4. **Forces docker-format images** (`export BUILDAH_FORMAT=docker`) so the `HEALTHCHECK` directive survives the build — Podman's default OCI format silently drops it, which once masked a crash-looping container (forensic anchor in the script comment, 2026-04-29).
5. **Delegates to the CLI** with `-cmd=start -project-dir=<repo> -profile=api-go` (plus `-with-observability` / `-dev-docs` when requested).

The `api-go` profile brings up three containers (from `docker-compose.yml`): `lava-postgres` (the database), `lava-migrate` (schema migration), and `lava-api-go` itself.

Why `network_mode: host`

In `docker-compose.yml` the `lava-api-go` service uses **`network_mode: host`**. This is required so the service's mDNS advertisement reaches the LAN — a bridged container cannot multicast to your Wi-Fi. The service listens on `:8443` (`LAVA_API_LISTEN`) for the public LAN listener (HTTP/3 + HTTP/2 over TLS).

Containers submodule

Generic container-runtime concerns live in the pinned upstream `vasic-digital/Containers` submodule (mounted at `submodules/containers/`). `start.sh` and `tools/lava-containers/` are **thin Lava-specific glue** that forward CLI flags; the heavy lifting (Podman/Docker detection, compose orchestration) is the submodule's job. Per the Containers-driven-emulators and no-host-direct rules, the container runtime is rootless Podman or Docker.

Stopping the stack

```
./stop.sh
```

Checking status

```
./tools/lava-containers/bin/lava-containers -cmd=status
```

This reports runtime, health, and the advertised IPs.

How the app discovers the local API (mDNS)

What gets advertised

`lava-api-go` advertises itself over **mDNS** (multicast DNS / Bonjour / zero-conf). The defaults come from `lava-api-go/internal/config/config.go`:

Config field	Env var	Default
`MDNSServiceType`	`LAVA_API_MDNS_TYPE`	`_lava-api._tcp`
`MDNSPort`	`LAVA_API_MDNS_PORT`	`8443`
`MDNSInstanceName`	`LAVA_API_MDNS_INSTANCE`	`Lava API`

A separate **dev** instance (for the `.dev`-suffixed debug app) can run side-by-side and advertises `_lava-api-dev._tcp` on port `8543` (`LAVA_API_DEV_*` in `.env.example`; bring it up via `docker compose -f docker-compose.dev.yml up`). Release builds ignore the dev service type entirely.

Service types and TXT records

From `docs/LOCAL_NETWORK_DISCOVERY.md`, the client resolves these:

Source	Service type	Key TXT records
Host server (release)	`_lava-api._tcp`	`engine=go`, `platform=server`
Host server (dev)	`_lava-api-dev._tcp`	`engine=go-dev`
On-device app (release)	`_lava-api._tcp`	`engine=go`, `platform=android`, `storage=sqlite`
On-device app (dev)	`_lava-api-dev._tcp`	`engine=go-dev`, `platform=android`

The Android client (`LocalNetworkDiscoveryServiceImpl`, in `core:data`) maps a discovered instance to `Endpoint.GoApi(host, port)` using the authoritative TXT `engine` attribute (falling back to the service type).

The discovery flow on the device

1. **Proxy/API advertises** when the service starts.
2. **App scans** — opening the connection/menu settings starts a network scan (timeout ~5 seconds).
3. **Manual refresh** — tap the refresh icon (↻) in the connection-settings bottom sheet to re-scan at any time.
4. **Auto-connect** — if an instance is found and you are not already on a custom mirror, the app adds it and opens the connection-settings dialog.
5. **Manual selection** — you can always pick an endpoint by hand.

> The on-device **advertiser** (a phone acting as the API) is **PENDING (Phase D)** — the client-side discovery described above is in-tree and works; the embed that registers a phone via `NsdManager` is not yet in the tree.

Authentication to a discovered API

Discovery only **finds** an instance; **connecting requires the Lava-Auth key**. Every `lava-api-go` instance enforces a Lava-Auth gate (HMAC-SHA256 over a base64 UUID credential in the `Lava-Auth` header, with per-IP backoff). The `/health` and `/ready` endpoints answer without a credential so discovery probes work, but any data endpoint returns `401` without the correct key.

Requirements

- Both the phone and the API host must be on the **same LAN / Wi-Fi**.
- Android 5.0+ (API 21+) for the app.
- The host must allow multicast (mDNS uses UDP port 5353). Most home routers do by default; some corporate networks block it — use manual endpoint entry there.

Troubleshooting

Issue	What to do
API not discovered	Tap the refresh icon in connection settings; confirm both devices share the network; check that mDNS (UDP 5353) is not firewalled.
Discovery slow	mDNS can take a few seconds; the scan timeout is ~5s.
Auto-connect didn't fire	If you already have a custom mirror set, the app will not auto-switch — select the discovered endpoint manually.
Data calls return 401	You found the instance but have no valid `Lava-Auth` key — supply the key for that instance.
Container "Up" but app can't reach it	Confirm `network_mode: host` and that `:8443` is listening; `./tools/lava-containers/bin/lava-containers -cmd=status` shows health + advertised IPs.